

Advanced Simulation Summative Coursework

MSc in Statistics 2025/26, Imperial College London

06051710

Filtering and smoothing for a nonlinear state space model

The SIR particle filter

1. Sampling

For this problem, I chose as my importance density $q(x_n|y_n, x_{n-1})$ an approximation of the optimal proposal (that is $p(x_n|y_n, x_{n-1})$), by linearisations similar to the extended Kalman Filter (EKF). Although a bootstrap proposal would be easier, I felt the density $g(y_n|x_n)$ was relatively informative (or ‘peaky’), which meant a more informative proposal seemed appropriate. Additionally, the notes suggest the EKF can give reasonable answers for models where the non-linearity is independent of the noise, which is the case for our model, as neither V_n or W_n appear in non-linear functions.

X_n can be linearly approximated as $X_n = a_n + A_n X_{n-1} + B_n Z_n$ (where $Z_n \sim N(0, 1)$) and Y_n can be approximated as $Y_n = c_n + C_n X_n + W_n$ (where $W_n \sim N(0, 1)$), both by a classic Taylor expansion. This leads to $B_n = \sqrt{0.1}$, $C_n = \frac{\psi_\theta(X_{n-1}, 0)}{10}$ and $D_n = 1$ (all other variables are not relevant for the proposal). The approximation of the optimal proposal¹ then takes the form $q(x_n|y_n, x_{n-1}) \propto N(m_n(x_{n-1}, y_n), S_n(x_{n-1}))$, where we have:

$$\begin{aligned} S_n^{-1}(x_{n-1}) &= (B_n B_n^T)^{-1} + C_n^T (D_n D_n^T)^{-1} C_n \\ S_n^{-1}(x_{n-1}) &= 10 + \frac{\psi_\theta(x_{n-1}, 0)^2}{100} \\ m_n(x_{n-1}, y_n) &= S_n(x_{n-1}) \left((B_n B_n^T)^{-1} \psi_\theta(x_{n-1}, 0) \right. \\ &\quad \left. + C_n^T (D_n D_n^T)^{-1} (y_n - \phi_\theta(\psi_\theta(x_{n-1}, 0), 0) + C_n \psi_\theta(x_{n-1}, 0)) \right) \\ m_n(x_{n-1}, y_n) &= S_n(x_{n-1}) \left(10 \psi_\theta(x_{n-1}, 0) + \frac{\psi_\theta(x_{n-1}, 0)}{10} \left(y_n + \frac{\psi_\theta(x_{n-1}, 0)^2}{20} \right) \right) \end{aligned}$$

¹this result is taken from the notes

For $q_\theta(x_0|y_0)$ the linearisation just leads to the bootstrap proposal, as linearising the likelihood of y_0 around the prior mean leads to $q_\theta(x_0|y_0) \sim N(0, 1)$ (i.e. the bootstrap in this case).

Using multinomial resampling and the above proposals then led to the following particle approximations for $p(x_n|y_{0:n})$ in Figure 1 and Figure 2, where the densities are estimated via the default ‘density’ function in R, which uses a Gaussian kernel and rule-of-thumb bandwidth selector based on the standard deviation of the data. The red dotted line represents the true value of the hidden state, demonstrating how the SIR particle filter performed relatively accurately, even with $N=50$.

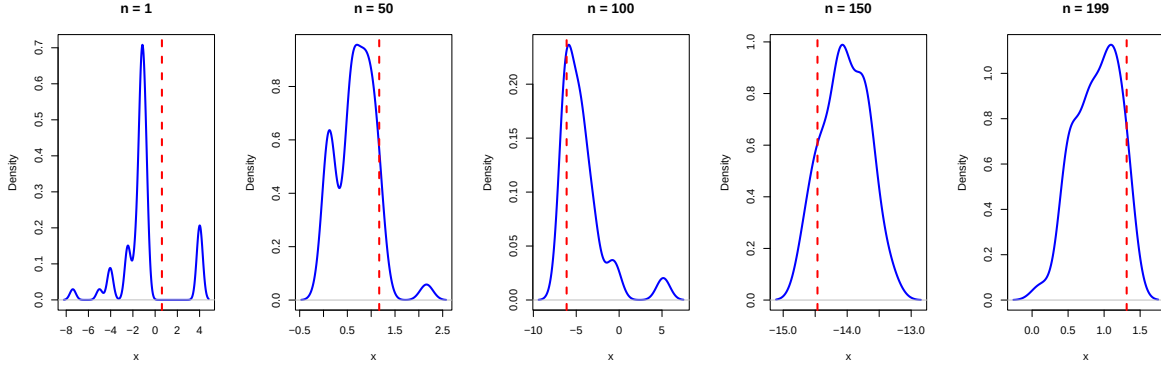


Figure 1: $N=50$ SIR particle filter approximations

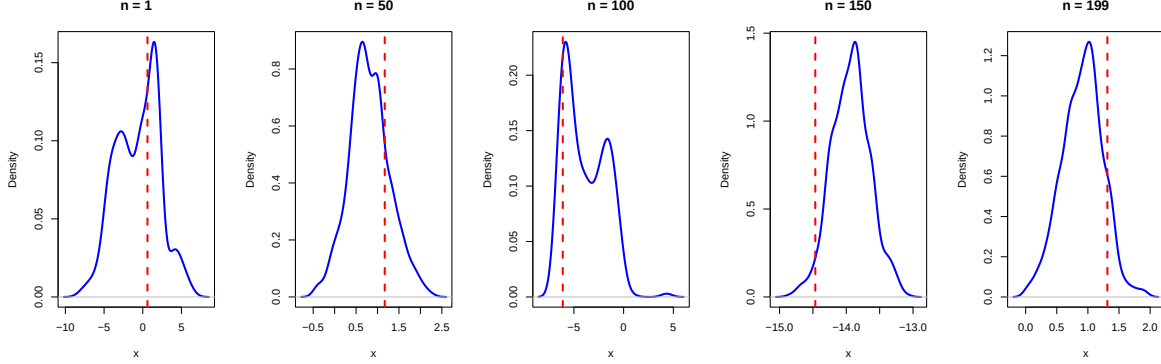


Figure 2: $N=500$ SIR particle filter approximations

2. Path Degeneracy

Plotting the final simulated trajectories $\bar{X}_{0:200}^i$ of the SIR algorithm in Figure 3 illustrates the path degeneracy phenomenon well.

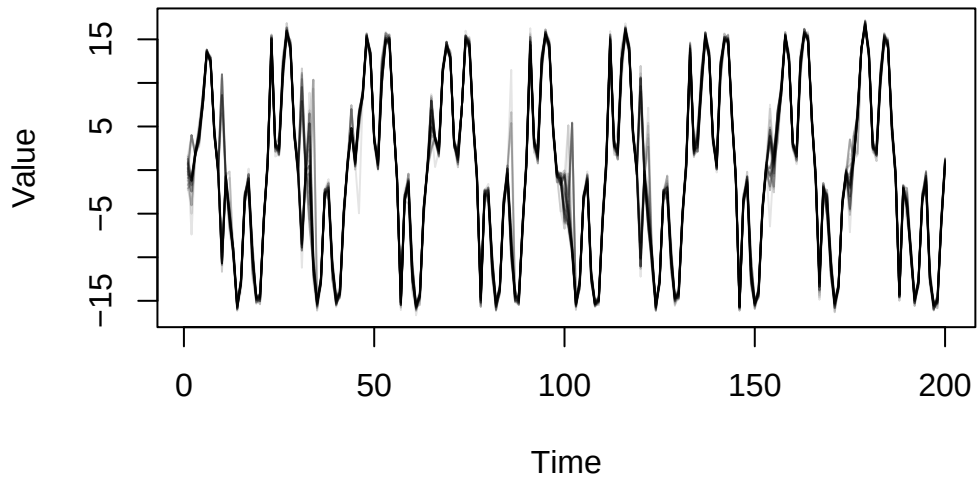


Figure 3: Path Degeneracy Plot

Code Appendix

```
knitr::opts_chunk$set(  
  collapse = TRUE,  
  comment = "#>"  
)  
include_solutions <- TRUE  
set.seed(8)  
require(rmarkdown)  
require(knitr)  
require(kableExtra)  
# Put any library imports and other preamble here.  
#Simulating the dataset  
x <- rep(0,201)  
y <- rep(0,201)  
x[1] <- rnorm(1)  
y[1] <- (x[1]^2)/20 + rnorm(1)  
  
for (i in 2:201){
```

```

  x[i] <- 0.5*x[i-1] + (25*x[i-1])/(1+(x[i-1])^2) + 8*cos(1.2*(i-1))+rnorm(1, mean=0, sd=sqr
  y[i] <- (x[i]^2)/20 + rnorm(1)
}
#A single forward run of a SIR particle filter

#creates a matrix with N rows and n columns
#each column will have the Xi sampled at time i
#weights will all be assumed to be 1/N, as the samples will be in-
#-the matrix only after resampling

psi0 <- function(x, n){
  return (0.5*x + (25*x)/(1+x^2) + 8*cos(1.2*n))
}

proposal <- function(x, y, n){
  psi <- psi0(x, n)
  Sn <- (10 + (psi^2)/100)^(-1)
  mn <- Sn * (10*psi + (psi/10) * (y + (psi^2)/20 ))
  return(list(mn = mn, Sn = Sn))
}

SIR <- function(N){
  samples <- matrix(rep(0,200*N), nrow=N, ncol=200)

  ## Sampling first samples

  samples0 <- rep(0,N)
  weights0 <- rep(0,N)

  for (i in 1:N){
    samples0[i] <- rnorm(1, mean=0, sd=1)
    #compute unnormalised weights- target over proposal
    #in bootstrap case this is just g
    weights0[i] <- dnorm(y[1], mean=(samples0[i]^2)/20, sd=1)
  }

  # normalise weights
  weights0 <- weights0/sum(weights0)

  #Resample (Multinomial Resampling)

  resamples <- sample(1:N, size = N, replace = TRUE, prob = weights0)

```

```

resamples0 <- rep(0,N)

for (i in 1:N){
  resamples0[i] <- samples0[resamples[i]]
}

#update matrix
samples[,1] <- resamples0

## Sampling remaining 199*N samples
for (k in 2:200){
  samplesk <- rep(0,N)
  weightsk <- rep(0,N)

  for (i in 1:N){
    Xkprev <- samples[,k-1][i]
    prop <- proposal(x=Xkprev, y=y[k], n=k-1)
    samplesk[i] <- rnorm(1, mean=prop$mn, sd=sqrt(prop$Sn))

    fx <- psi0(Xkprev, k-1)
    transition <- dnorm(samplesk[i], mean=fx, sd=sqrt(0.1))
    likelihood <- dnorm(y[k], mean=(samplesk[i]^2)/20, sd=1)
    proposal_density <- dnorm(samplesk[i], mean=prop$mn, sd=sqrt(prop$Sn))

    log_w <- log(transition) + log(likelihood) - log(proposal_density)
    weightsk[i] <- exp(log_w)
  }
  #normalise weights
  weightsk <- weightsk/sum(weightsk)

  #Multinomial Resampling
  resamples <- sample(1:N, size=N, replace=TRUE, prob=weightsk)
  resamplesk <- rep(0,N)

  for (i in 1:N){
    resamplesk[i] <- samplesk[resamples[i]]
  }
}

```

```

    #update matrix
    samples[,k] <- resamplesk
  }
  return (samples)
}

sir50 <- SIR(50)
sir500 <- SIR(500)
times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(sir50[, n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
}

times = c(2,51,101,151,200)
par(mfrow = c(1, length(times)))

for (n in times) {
  d <- density(sir500[, n])
  plot(d, lwd = 2, col = "blue",
       main = paste("n =", n-1),
       xlab = "x", ylab = "Density")
  abline(v = x[n], col = "red", lwd = 2, lty = 2)
}

matplot(t(sir50), type = "l", lty = 1,
       col = rgb(0, 0, 0, 0.1),
       xlab = "Time", ylab = "Value")

```